CNet 2 notes

Friday, July 3, 2026 11:38 AM

Layer (abstract base class)

fields
- input/output dimension
  - As tuple or XTensor internal dimension struct?
- integer ID, assigned at addition to network
- Concrete impl's — weights + gradients

methods
- getters
  - id (void → int)
    - No setter. User should not manipulate ID.
  - input + output dimension (void → tuple of some kind)
- others
  - forward
  - predict
    - Forward without storing gradients
  - reverse
    - Backpropagation operation

→ Perhaps have base-class have these methods as pure virtual methods

Then each concrete implementation (i.e. Linear, Convolutional 2D, Splitter Adapter) implements methods

Not a true layer, but dictates network structure.

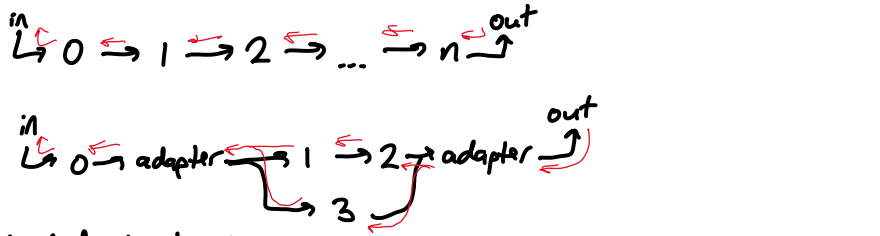
Forward / reverse computation —

Must compute ONE forward operation at a time
- Assign sequence to each layer

Test !

LayerGraph. Stores network architecture as Layer pointers

ex.

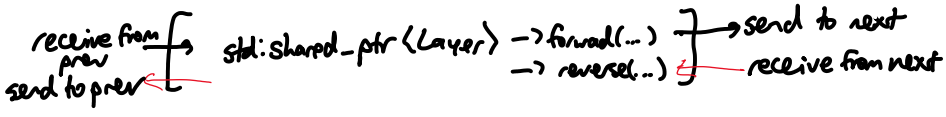


Linked structure: (Forward + reverse pointers)

Enable-OK check:
Recursively check all paths through network.
Dimension mismatch → Throw exception!
Multiple ends → Throw exception!

Node types
- Single: forward + reverse only

Process of forward or reverse operation



- Adapter: multiple inputs or outputs On Hold for combine operation
  - splitter: one input → many outputs
    ex. $\begin{bmatrix} x & x & x \\ x & x & x \end{bmatrix} \rightarrow \begin{bmatrix} x & x \\ x & x \end{bmatrix}, \begin{bmatrix} x \\ x \end{bmatrix}$ (to (0), to (1))
  - combiner: many inputs → one output
    ex. $\begin{bmatrix} x \\ x \end{bmatrix}, \begin{bmatrix} x \\ x \end{bmatrix}, \begin{bmatrix} x \\ x \end{bmatrix} \rightarrow \begin{bmatrix} x & x & x \\ x & x & x \end{bmatrix}$ (0) (1) (2)

Computation
1. Make std::vector of intermediate outputs
   Also store pointers to next node. init to first Layer
2. Compute output for intermediate output index 0
   If layer is:
   - Single - pass to next layer
     store output in intermediate outputs [0]

If layer is:

- single - pass to next layer
  Store output in intermediate outputs [0]
- adapter splitter -
  put each output in intermediate out's
- adapter combiner -
  have combiner count number of inputs received, and organize them by shape
  When all inputs received, pass to inter out's [0] for next layer
  Then reset combiner's N inputs received

Add layer operation

Need predecessor + new layer.

Must maintain pointers to all terminal Layer nodes.

Remove layer operation

Remove end, and ≥2 terminal nodes - throw exception! Must be unique end node.

If adapter:

Precondition - adapter cannot have successor, i.e. adapter must be terminal node

Convert connected Layers to terminal nodes

If single:

Remove in O(1) op.

By name or ID

First check terminal nodes.

Then: Recursively search graph

Prefer BFS. Adapter bypasses are likely short

LayerVector

ex.

index = order of computation

next index

0 : 1
1 : 2, 3 → Store each output individually
2 : 4
3 : 5
4 : 6
5 : 6
6 : out

3

not likely to be used

Using tensors directly (used in prod libraries, i.e. Pytorch) **Preferred**

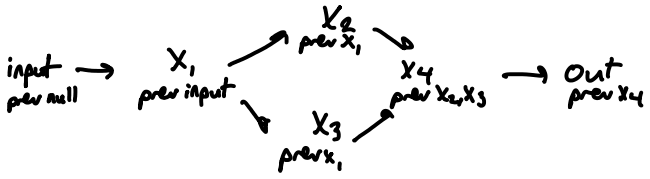
Wrap tensors, point to operators

Operator: does calculation on tensor
i.e. +, ÷, matrix multiplication, convolutional pass

TensorNode

Fields
- data (xt::xarray): tensor stored inside
- gradients (xt::xarray, same shape as the data): for backpropagation
- prev_operation (std::shared_ptr to a tensor operation): operation that created the tensor node

Methods
- Getters and setters for prev_operation (keep setter Protected)



Tensor Operation

Represents an operation that can be done on a tensor node (i.e. linear, addition, sine, convolutional 2d)

Fields
- predecessors (std::vector of std::shared_ptr's to TensorNodes): list of tensor nodes that come before
- successors (std::vector of **std::weak_ptr's** to TensorNodes): list of tensor nodes that come after. THESE ARE WEAK POINTERS!!! If shared pointers are used, the resulting dependency loop results in memory never being freed.

Methods
- forward: compute forward operation
  - Must also wrap the output in a tensor node, making it track its previous operation
- predict: compute forward operation. Don't record previous operation
- reverse: compute backwards pass
  - Also gives references to the predecessor operation node(s)

Overall

input → Wrap input in TensorNode

→ Tensor Op. (increment)
- Increment the tensor
- Record the initial input tensor node as its predecessor

→ To be continued...